

INTRODUCTION

The Public Health Clinic Records System is a database solution developed to support the efficient management of healthcare information within public health facilities. The system provides a structured method for storing and managing patient records, healthcare worker information, appointments, clinic visits, diagnoses, and treatment records.

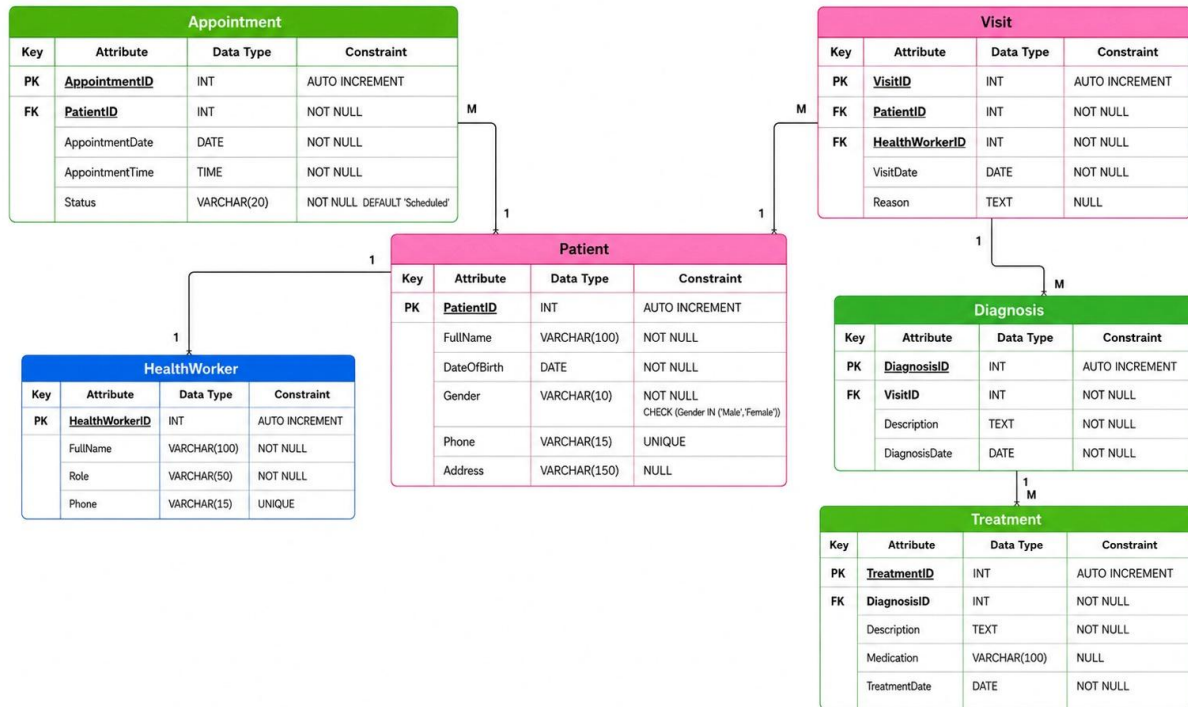
Many healthcare institutions face challenges in maintaining accurate patient records due to manual record-keeping processes. Such challenges may lead to misplaced files, delays in accessing patient information, and difficulties in monitoring patient treatment history. The implementation of a database system helps address these issues by providing a reliable platform for data storage and retrieval.

This project supports Sustainable Development Goal (SDG) 3: Good Health and Well-being by promoting effective healthcare information management and improving service delivery in public health clinics across Sierra Leone.

The objective of this project is to design and implement a relational database using MySQL that enables healthcare facilities to manage patient information efficiently while ensuring data integrity, consistency, and security.

REFINED PHYSICAL DATA MODEL

PHYSICAL DATA MODEL – HEALTH CLINIC RECORDS SYSTEM



The Health Clinic Records System consists of six entities that work together to manage healthcare operations within a clinic.

Patient

The Patient entity stores personal information about individuals receiving healthcare services.

Attributes

- PatientID (Primary Key)
- FullName
- DateOfBirth
- Gender
- Phone
- Address

HealthWorker

The HealthWorker entity stores information about medical personnel responsible for patient care.

Attributes

- HealthWorkerID (Primary Key)
- FullName
- Role
- Phone

Appointment

The Appointment entity manages patient appointment schedules.

Attributes

- AppointmentID (Primary Key)
- PatientID (Foreign Key)
- AppointmentDate
- AppointmentTime
- Status

Visit

The Visit entity records patient visits to the clinic.

Attributes

- VisitID (Primary Key)
- PatientID (Foreign Key)
- HealthWorkerID (Foreign Key)
- VisitDate
- Reason

Diagnosis

The Diagnosis entity records medical diagnoses made during patient visits.

Attributes

- DiagnosisID (Primary Key)
- VisitID (Foreign Key)
- Description
- DiagnosisDate

Treatment

The Treatment entity stores treatment details prescribed after diagnosis.

Attributes

- TreatmentID (Primary Key)
- DiagnosisID (Foreign Key)
- Description
- Medication
- Treatment Date

Relationships

- One Patient can have many Appointments.
- One Appointment belongs to one Patient.
- One Patient can have many Visits.
- One Visit belongs to one Patient.
- One Health Worker can attend many Visits.
- One Visit is attended by one Health Worker.
- One Visit can have many Diagnoses.
- One Diagnosis belongs to one Visit.
- One Diagnosis can have many Treatments.
- One Treatment belongs to one Diagnosis.

EXPLANATION OF DESIGN DECISIONS

The database was designed using relational database principles to ensure efficient management of healthcare information while minimizing data redundancy.

Primary Keys

Each table contains a primary key that uniquely identifies records within the database. Auto-incrementing integer values were selected because they provide efficient indexing and simplify record creation.

Foreign Keys

Foreign keys were implemented to establish relationships among tables and maintain referential integrity.

Examples include:

- PatientID links Appointment records to Patients.
- PatientID links Visit records to Patients.
- HealthWorkerID links Visits to Health Workers.
- VisitID links Diagnoses to Visits.
- DiagnosisID links Treatments to Diagnoses. Data Types

Appropriate data types were selected according to the nature of the data being stored.

- INT was used for identifiers.
- VARCHAR was used for names, phone numbers, roles, and medications.
- DATE was used for appointment, visit, diagnosis, and treatment dates.
- TIME was used for appointment scheduling.
- TEXT was used for descriptions and medical notes.

Constraints

Several constraints were implemented to improve data quality.

- NOT NULL ensures mandatory fields contain values.
- UNIQUE prevents duplicate phone numbers.
- AUTO_INCREMENT generates unique identifiers automatically.
- CHECK constraints validate gender values.

These design decisions help maintain consistency, reliability, and accuracy throughout the database.

SQL IMPLEMENTATION

SQL Code for Database Creation

```
CREATE DATABASE Public_Health_Clinic_records
```

SQL Code for Database Table Creation

APPOINTMENT

```
CREATE TABLE Appointment (  
    AppointmentID INT AUTO_INCREMENT PRIMARY KEY,  
    PatientID INT NOT NULL,  
    AppointmentDate DATE NOT NULL,  
    AppointmentTime TIME NOT NULL,  
    Status VARCHAR(20) NOT NULL DEFAULT 'Scheduled',  
    FOREIGN KEY (PatientID) REFERENCES Patient(PatientID)  
);|
```

PATIENT

```
|CREATE TABLE Patient (  
    PatientID INT AUTO_INCREMENT PRIMARY KEY,  
    FullName VARCHAR(100) NOT NULL,  
    DateOfBirth DATE NOT NULL,  
    Gender VARCHAR(10) NOT NULL CHECK (Gender IN ('Male', 'Female')),  
    Phone VARCHAR(15) UNIQUE,  
    Address VARCHAR(150)  
);
```

HEALTHWORKER

```
|CREATE TABLE HealthWorker (  
    HealthWorkerID INT AUTO_INCREMENT PRIMARY KEY,  
    FullName VARCHAR(100) NOT NULL,  
    Role VARCHAR(50) NOT NULL,  
    Phone VARCHAR(15) UNIQUE  
);
```

VISIT

```
CREATE TABLE Visit (  
    VisitID INT AUTO_INCREMENT PRIMARY KEY,  
    PatientID INT NOT NULL,  
    HealthWorkerID INT NOT NULL,  
    VisitDate DATE NOT NULL,  
    Reason TEXT,  
    FOREIGN KEY (PatientID) REFERENCES Patient(PatientID),  
    FOREIGN KEY (HealthWorkerID) REFERENCES HealthWorker(HealthWorkerID)  
);
```

TREATMENT

```
CREATE TABLE Treatment (  
    TreatmentID INT AUTO_INCREMENT PRIMARY KEY,  
    DiagnosisID INT NOT NULL,  
    Description TEXT NOT NULL,  
    Medication VARCHAR(100),  
    TreatmentDate DATE NOT NULL,  
    FOREIGN KEY (DiagnosisID) REFERENCES Diagnosis(DiagnosisID)  
);
```

DIAGNOSIS

```
CREATE TABLE Diagnosis (  
    DiagnosisID INT AUTO_INCREMENT PRIMARY KEY,  
    VisitID INT NOT NULL,  
    Description TEXT NOT NULL,  
    DiagnosisDate DATE NOT NULL,  
    FOREIGN KEY (VisitID) REFERENCES Visit(VisitID)  
);
```


SQL CODE FOR DATA MANIPULATION

Samples were inserted into database for testing the accuracy.

Diagnosis

```
INSERT INTO Diagnosis (VisitID, Description, DiagnosisDate)
VALUES (1, 'Normal health condition', '2026-06-10'),
       (2, 'Malaria detected', '2026-06-11'),
       (3, 'Malaria detected', '2026-07-21'),
       (4, 'Typhoid Detected', '2026-08-19'),
       (5, 'Cold Detcted', '2026-08-25')
```

Patient

```
INSERT INTO Patient (FullName, DateOfBirth, Gender, Phone, Address)
VALUES ('John Kamara', '1990-05-12', 'Male', '23276123456', 'Freetown'),
       ('Fatmata Sesay', '1985-09-20', 'Female', '23276123457', 'Bo'),
       ('Maybel Sesay', '1989-09-27', 'Female', '23276268533', 'Grafton'),
       ('John Conteh', '1990-05-10', 'male', '23276567864', 'Kissy'),
       ('Philp Kanu', '2000-02-19', 'male', '23277654323', 'Calaba Town')
```

Treatment

```
1 INSERT INTO Treatment (DiagnosisID, Description, Medication, TreatmentDate)
2 VALUES (2, 'Antimalarial treatment', 'Coartem', '2026-06-11'),
3         (3, 'Antimalarial treatment', 'Coartem', '2026-07-21'),
4         (4, 'Typhoid treatment', 'Cefixime', '2026-08-19'),
5         (5, 'Cold treatment', 'Ibuprofen', '2026-08-25')
6
7
8
9
10
```

Health worker

```
INSERT INTO HealthWorker (FullName, Role, Phone)
VALUES ('Dr. Mariatu Conteh', 'Doctor', '23276123458'),
       ('Nurse Abdul Koroma', 'Nurse', '23276123459'),
       ('Nurse Alphina Conteh', 'Nurse', '23278765432'),
       ('Dr Esther Kargbo', 'Doctor', '23279876543'),
       ('Dr. Zainiel Smart', 'Doctor', '232734454321')
```

Visit

```
INSERT INTO Visit (PatientID, HealthWorkerID, VisitDate, Reason)
VALUES (1, 1, '2026-06-10', 'Routine Checkup'),
       (2, 2, '2026-06-11', 'Fever and headache'),
       (3, 3, '2026-07-21', 'Fever '),
       (4, 4, '2026-08-19', 'Stomach pain'),
       (5, 5, '2026-08-25', 'Toothache')
```

Appointment

```
INSERT INTO Appointment (PatientID, AppointmentDate, AppointmentTime)
VALUES (1, '2026-06-10', '09:00:00'),
       (2, '2026-06-11', '08:35:00'),
       (3, '2026-07-21', '10:45:00'),
       (4, '2026-08-19', '02:00:00'),
       (5, '2026-08-25', '12:50:00')
```

Demonstration of UPDATE Statement

```
UPDATE Patient  
SET Phone='079123456'  
WHERE PatientID=1;
```

You have a previously saved query. Click the auto-saved query to load the query.

The UPDATE statement is used to modify existing records without affecting other records stored in the table.

Demonstration of DELETE Statement

```
DELETE FROM Appointment  
WHERE AppointmentID=2;
```

The DELETE statement removes unwanted records from the database when they are no longer required.

SQL QUERIES

SELECT Queries

```
SELECT * FROM Patient;
```

✓ Showing rows 0 - 4 (5 total, Query took 0.0004 seconds.)

`SELECT * FROM Patient;`

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

☐ Show all | Number of rows: 25 | Filter rows: Sort by key: None

Extra options

		PatientID	FullName	DateOfBirth	Gender	Phone	Address		
☐	Edit	Copy	Delete	1	John Kamara	1990-05-12	Male	079123456	Freetown
☐	Edit	Copy	Delete	2	Fatmata Sesay	1985-09-20	Female	23276123457	Bo
☐	Edit	Copy	Delete	3	Maybel Sesay	1989-09-27	Female	23276268533	Grafton
☐	Edit	Copy	Delete	4	John Conteh	1990-05-10	male	23276567864	Kissy
☐	Edit	Copy	Delete	5	Philp Kanu	2000-02-19	male	23277654323	Calaba Town

↑ ☐ Check all With selected: [Edit](#) [Copy](#) [Delete](#) [Export](#)

☐ Show all | Number of rows: 25 | Filter rows: Sort by key: None

Query results operations

[Print](#) [Copy to clipboard](#) [Export](#) [Display chart](#) [Create view](#)

[Bookmark this SQL query](#)

Label: ☐ Let every user access this bookmark

The SELECT statement retrieves information stored in database tables.

Filtering Using WHERE

```
SELECT * FROM Patient
WHERE Gender='Female';
```

Show query box

✓ Showing rows 0 - 1 (2 total, Query took 0.0006 seconds.)

```
SELECT * FROM Patient WHERE Gender='Female';
```

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

		PatientID	FullName	DateOfBirth	Gender	Phone	Address		
☐	Edit	Copy	Delete	2	Fatmata Sesay	1985-09-20	Female	23276123457	Bo
☐	Edit	Copy	Delete	3	Maybel Sesay	1989-09-27	Female	23276268533	Grafton

☐ Check all | With selected: Edit Copy Delete Export

The WHERE clause filters records according to a specified condition.

Sorting Using ORDER BY

```
SELECT * FROM Appointment
ORDER BY AppointmentDate ASC;
```

✓ Showing rows 0 - 4 (5 total, Query took 0.0007 seconds.) [AppointmentDate: 2026-06-10... - 2026-08-25...]

```
SELECT * FROM Appointment ORDER BY AppointmentDate ASC;
```

☐ Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

☐ Show all

Number of rows: 25

Filter rows: Search this table

Sort by key: None

Extra options

				AppointmentID	PatientID	AppointmentDate	AppointmentTime	Status
☐	 Edit	 Copy	 Delete	1	1	2026-06-10	09:00:00	Scheduled
☐	 Edit	 Copy	 Delete	2	2	2026-06-11	08:35:00	Scheduled
☐	 Edit	 Copy	 Delete	3	3	2026-07-21	10:45:00	Scheduled
☐	 Edit	 Copy	 Delete	4	4	2026-08-19	02:00:00	Scheduled
☐	 Edit	 Copy	 Delete	5	5	2026-08-25	12:50:00	Scheduled

☐ Check all With selected: Edit Copy Delete Export

The ORDER BY clause organizes records in ascending or descending order.

Aggregate Functions

COUNT

```
1 SELECT COUNT(*) AS TotalPatients
2 FROM Patient;
```

Your SQL query has been executed successfully.

```
SELECT COUNT(*) AS TotalPatients FROM Patient;
```

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

Extra options

TotalPatients

5

Query results operations



Print



Copy to clipboard



Export



Display chart



Create view

Count calculates the total number of records.

AVERAGE

```
SELECT AVG(TIMESTAMPDIFF(YEAR, DateOfBirth, CURDATE()))  
AS AverageAge  
FROM Patient;  
|
```

✓ Showing rows 0 - 0 (1 total, Query took 0.0110 seconds.)

```
SELECT AVG(TIMESTAMPDIFF(YEAR, DateOfBirth, CURDATE())) AS AverageAge FROM Patient;
```

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

☐ Show all | Number of rows: 25 ▾ Filter rows:

Extra options

AverageAge

34.8000

☐ Show all | Number of rows: 25 ▾ Filter rows:

Query results operations

[Print](#) [Copy to clipboard](#) [Export](#) [Display chart](#) [Create view](#)

[Bookmark this SQL query](#)

Label: ☐ Let every user access this bookmark

[Bookmark this SQL query](#)

Average calculates the average value of a numeric expression.

LIMIT Clause

```
SELECT * FROM Visit  
LIMIT 5;
```

✓ Showing rows 0 - 4 (5 total, Query took 0.0006 seconds.)

```
SELECT * FROM Visit LIMIT 5;
```

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

Extra options

				VisitID	PatientID	HealthWorkerID	VisitDate	Reason
☐					1	1	1 2026-06-10	Routine Checkup
☐					2	2	2 2026-06-11	Fever and headache
☐					3	3	3 2026-07-21	Fever
☐					4	4	4 2026-08-19	Stomach pain
☐					5	5	5 2026-08-25	Toothache

↑ ☐ Check all With selected: Edit Copy Delete Export

LIMIT restricts the number of rows returned by a query.

USER MANAGEMENT

Creation of Users

```
1 CREATE USER 'Esther'@'localhost'  
2 IDENTIFIED BY '905005763';  
3 CREATE USER 'Mamadu'@'localhost'  
4 IDENTIFIED BY '905005082';  
5 CREATE USER 'Amadu'@'localhost'  
6 IDENTIFIED BY '905005800';
```

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0136 seconds.)

```
CREATE USER 'Esther'@'localhost' IDENTIFIED BY '905005763';
```

[Edit inline] [Edit] [Create PHP code]

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0015 seconds.)

```
CREATE USER 'Mamadu'@'localhost' IDENTIFIED BY '905005082';
```

[Edit inline] [Edit] [Create PHP code]

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0016 seconds.)

```
CREATE USER 'Amadu'@'localhost' IDENTIFIED BY '905005800';
```

[Edit inline] [Edit] [Create PHP code]

Granting Privileges

```
1 GRANT ALL PRIVILEGES
2 ON Public_Health_Clinic_Records.*
3 TO 'Esther'@'localhost';
4 GRANT ALL PRIVILEGES
5 ON Public_Health_Clinic_Records.*
6 TO 'Mamadu'@'localhost';
7 GRANT ALL PRIVILEGES
8 ON Public_Health_Clinic_Records.*
9 TO 'Amadu'@'localhost';
```

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0043 seconds.)

```
GRANT ALL PRIVILEGES ON Public_Health_Clinic_Records.* TO 'Esther'@'localhost';
```

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

⚠ Error: #1046 No database selected

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0409 seconds.)

```
GRANT ALL PRIVILEGES ON Public_Health_Clinic_Records.* TO 'Mamadu'@'localhost';
```

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

⚠ Error: #1046 No database selected

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0020 seconds.)

```
GRANT ALL PRIVILEGES ON Public_Health_Clinic_Records.* TO 'Amadu'@'localhost';
```

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

⚠ Error: #1046 No database selected

IMPORTANCE OF DATABASE SECURITY

Database security is essential in healthcare systems because patient information is highly sensitive and confidential. Unauthorized access to medical records may compromise patient privacy and affect the quality of healthcare services.

Security measures such as user authentication, password management, and access privileges help ensure that only authorized personnel can access or modify records. These controls protect data integrity, confidentiality, and availability while reducing the risk of data loss and cyber threats.

Effective database security also promotes trust between healthcare providers and patients by ensuring that personal medical information remains protected.

CONCLUSION

The Public Health Clinic Records System was successfully designed and implemented using MySQL based on the refined physical data model. The system demonstrates the practical application of database concepts including table creation, relationships, constraints, data manipulation, query execution, and user management.

The database provides an organized approach to managing patient information, healthcare workers, appointments, visits, diagnoses, and treatments. Through efficient data management and improved accessibility of healthcare records, the system contributes to better service delivery within public health facilities.

Overall, the project achieved its objectives and highlights the importance of database systems in supporting healthcare management and improving public health services in Sierra Leone.

REFERENCES

- Elmasri, R., & Navathe, S. B. (2016). Fundamentals of Database Systems. Pearson Education.
- Connolly, T., & Begg, C. (2015). Database Systems: A Practical Approach to Design, Implementation and Management. Pearson.
- Oracle Corporation. (2024). MySQL Reference Manual. Available at: <https://dev.mysql.com/doc/>